

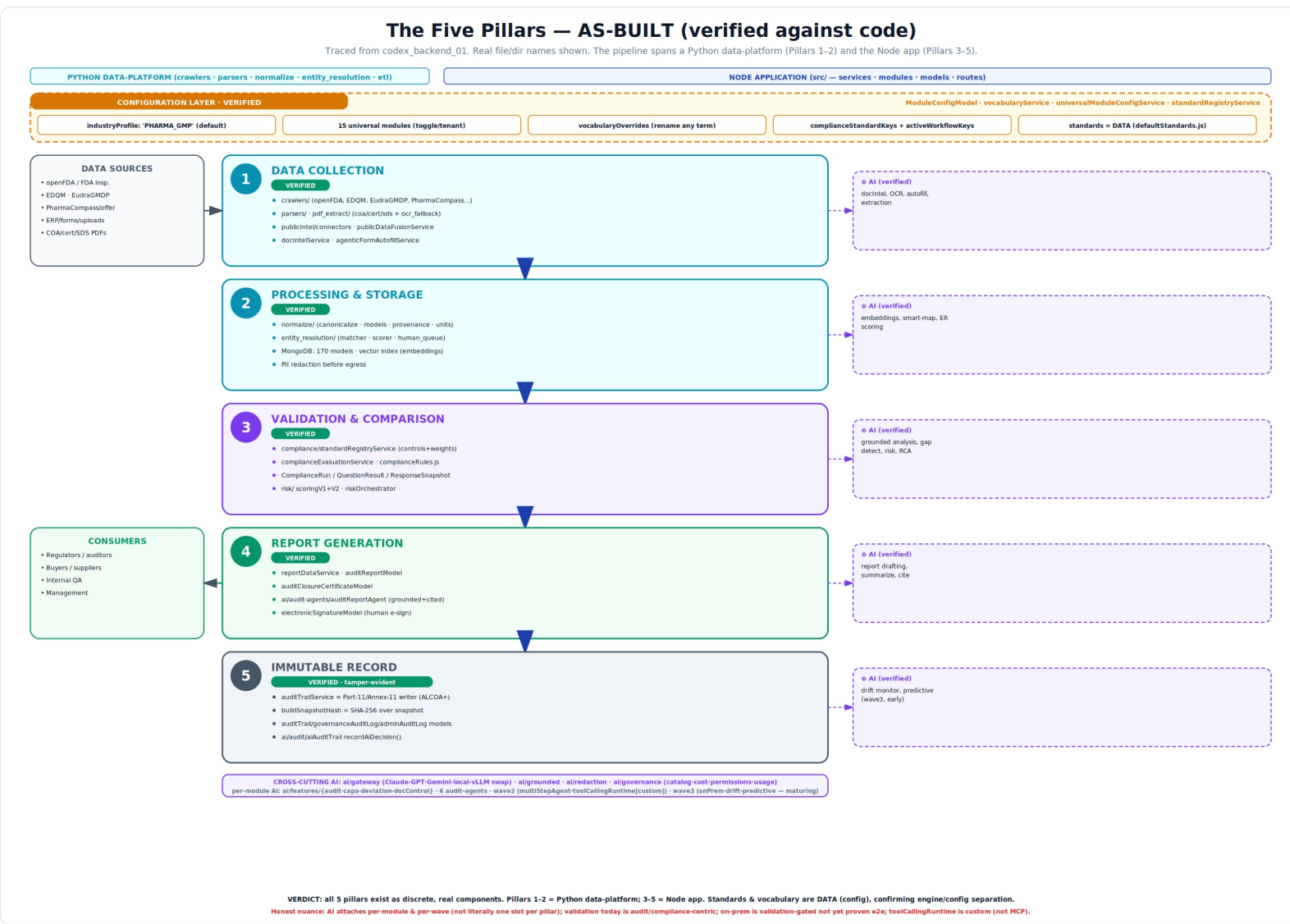
The five-pillar architecture — verified, as-built vs. to-be

This revision is traced directly from the cloned source (codex_backend_01). Claims are now backed by real file and directory names, not reconstruction. It also shows the as-built reality vs. the idealized model, and how every module shares one data model and architecture.

How this was verified. The backend repository was cloned and inspected this session. Every pillar below cites the actual files/directories that implement it. Where the as-built reality differs from the clean five-pillar picture, that difference is stated plainly rather than smoothed over. This replaces the earlier "conceptual model pending verification" with a code-checked account.

Verdict in one line

The five-pillar model is substantially confirmed. All five pillars exist as discrete, real components, and the engine/configuration separation that makes the platform industry-agnostic is real (standards and vocabulary are stored as data, not hard-coded). Two honest refinements: the pipeline spans **two runtimes** (a Python data-platform for collection+normalization, the Node app for validation→report→record), and AI attaches **per-module** and **per-wave** rather than as one literal slot per pillar. Neither weakens the model; both make it more accurate.



As-built: real file/directory names traced from codex_backend_01. Python data-platform (Pillars 1-2) → Node app (Pillars 3-5), with the verified configuration layer on top and AI gateway cross-cutting.

What each pillar actually is, in code

Pillar	Status	Implementing files / directories (actual)	What it does
1 • Collection	VERIFIED	crawlers/ (openfda_ndc, edqm_cep, eudragmdp, pharmacompass, pharmaoffer, cphi), parsers/, pdf_extract/ (coa_parser, cert_parser, sds_parser, ocr_fallback), publicIntel/connectors , docIntelService.js , agenticFormAutofillService.js	Structured connectors + unstructured PDF/OCR + public-data fusion + form autofill.
2 • Processing & Storage	VERIFIED	normalize/ (canonicalize, models, provenance, units, validators), entity_resolution/ (matcher, scorer, conflicts, human_queue), 170 Mongoose models, vector services (askHawkEmbeddingService , complianceGuidelineVectorService)	Canonicalize to a common model, resolve entities (with human queue), store + index.
3 • Validation	VERIFIED	compliance/standardRegistryService.js , complianceEvaluationService.js , complianceRules.js (evaluateQuestionCompliance), risk/ (scoringV1/V2, riskOrchestrator); models: ComplianceRun, ComplianceQuestionResult, ComplianceResponseSnapshot	Load configured standard's controls → compare → verdicts + risk weighting.
4 • Report	VERIFIED	reportDataService.js , ai/audit-agents/auditReportAgent.js , models: auditReportModel, auditClosureCertificateModel, electronicSignatureModel	Assemble cited report/certificate → human e-sign.
5 • Immutable Record	VERIFIED (tamper-evident)	auditTrailService.js (Part-11/Annex-11 writer, ALCOA+), buildSnapshotHash = crypto.createHash("sha256") , models: auditTrail, governanceAuditLog, adminAuditLog; ai/audit/aiAuditTrail.recordAiDecision()	Hash snapshot + append-only ALCOA+ trail + AI decision record.

The one wording correction the code forces. Pillar 5 is **tamper-evident**: per-record SHA-256 hashing ([buildSnapshotHash](#)) plus an append-only audit trail. There is **no chained ledger / blockchain** in the code. "Immutable record" in the honest sense = tamper-evident + append-only. Do not say "blockchain."

The two standing questions — now answered from code

Question	Answer (from source)
Is the tool runtime MCP-compliant?	No — custom. wave2/toolCallingRuntime.js has no MCP protocol/handshake. The gateway comment references an "MCP path" for local models as a label, but it is not a Model Context Protocol implementation. Say "custom tool-calling," not "MCP."
Has on-prem LLM run end-to-end?	Built & validation-gated, not proven. wave3/onPremLlmDeploy.js has registerOnPremEndpoint() and a frozen ONPREM_VALIDATION_REQUIREMENTS (IQ/OQ/PQ, change control, DR, model card). It's real scaffolding with the validation bar defined — say "available, validation-gated," not "proven in production."

The engine/configuration separation is real

The claim that one engine serves many industries rests on standards and vocabulary being **data, not code**. The source confirms this directly.

The configuration layer, in code

Config mechanism (file)	What it proves
<code>vocabularyService.js</code>	Per-tenant vocabulary overrides — "audit", "supplier", "buyer", "finding", "capa" etc. are all renameable per tenant with a defaults fallback. The UI language is configuration.
<code>universalModuleConfigService.js</code>	An explicit <code>industryProfile</code> field (default <code>'PHARMA_GMP'</code>), 15 universal modules toggled per tenant, dependency bundles, and <code>complianceStandardKeys</code> + <code>activeWorkflowKeys</code> per tenant.
<code>modules/compliance/defaultStandards.js</code>	Standards defined as data objects — e.g. <code>ICH_Q7_CFR21</code> with a list of controls (<code>controlId</code> , <code>clauseRef</code> , <code>standardRefs</code> , <code>keywords</code> , <code>expectedAnswer</code> , <code>weight</code>). A new standard is a new data object, not new code.
<code>standardRegistryService.js</code>	Loads, sanitizes, and versions those standards into the validation engine — the registry that lets you add/swap a standard at runtime.
<code>modules/auditEngine/modulePacks.js</code>	<code>MODULE_PACKS</code> (e.g. cGMP/ICH Q7) define phases, milestones, and pre-audit questions as data per module type — the audit workflow itself is configuration.
<code>WorkflowDefinitionService.js</code>	Tenant-scoped, versioned workflow definitions loaded by <code>workflowKey</code> — the lifecycle is data, resolved per tenant.

The test passes. Adding a new industry should require a new configuration pack (vocabulary + standards + workflow + module toggles), not edits to the pipeline. The code shows exactly these as data-driven, tenant-resolved structures. The engine/configuration separation is genuine — this is what makes "industry-agnostic by design" an accurate claim rather than a hope.

The honest boundary. `industryProfile` defaults to `PHARMA_GMP` and the shipped standards library is pharma (ICH Q7 + 21 CFR). So the engine is industry-agnostic **by construction**, and **pharma is the instantiated, proven vertical**; other industries are supported config slots, not yet populated. Accurate phrasing: "industry-agnostic engine, pharma proven, other verticals are configuration work on the roadmap."

Actual today vs. the aspirational picture

The earlier diagram was an idealized model. Here is the honest gap between what the code does today (as-is) and the fuller vision (to-be), pillar by pillar.

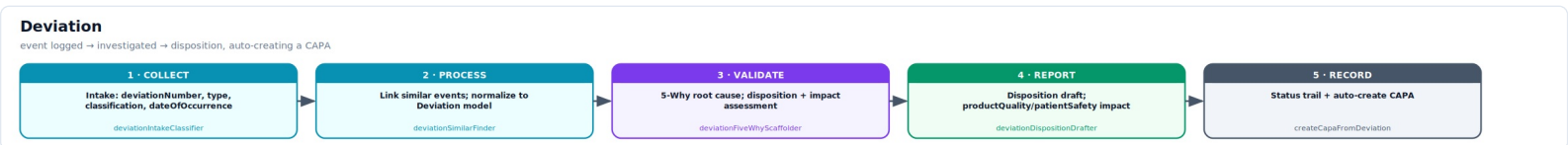
Pillar	AS-IS (in code today)	TO-BE (vision / partial)
1 • Collection	Public-data crawlers, PDF/OCR parsers, form autofill, doc intel — working.	Broader live IoT/sensor ingestion (wave3 IoTEquipmentFusion is an early stub); more connectors per industry.
2 • Processing	Python normalize + entity resolution with human queue; Mongo + vectors.	Tighter unification of the Python data-platform and Node model layer into one seamless common-model surface.
3 • Validation	Compliance run over audit questionnaires vs standard controls; risk scoring. Genuine rules engine.	Generalize the same engine to arbitrary batch/sensor/spec data beyond the audit-questionnaire flow.
4 • Report	Audit reports + closure certificates, grounded & cited, e-signed.	Wider template library per industry; richer certificate types.
5 • Record	SHA-256 snapshot hash + append-only Part-11 trail + AI decision record.	Optional chained/cross-party ledger for multi-party trust (today: per-record hashing only — not a chain).
AI	Gateway (Claude/GPT/Gemini/local) + grounded + redaction + governance; per-module drafters; 6 audit agents; wave2 multi-step.	Wave3 maturing: on-prem proven e2e, drift monitoring, predictive CAPA, IoT fusion. <code>toolCallingRuntime</code> → standard protocol (e.g. MCP) if desired.
Verticals	Pharma GMP fully instantiated (ICH Q7 + 21 CFR).	Food/HACCP, device/ISO 13485, etc. as new config packs.

The as-is is stronger than a typical "AI wrapper." What the code shows is a real data-platform + a real rules/validation engine + a real Part-11 audit substrate + a genuine config layer — with AI plugged in on top. The to-be is mostly **breadth** (more verticals, more data types, wave3 maturity), not missing foundations. That's a healthy gap: the fundamentals exist; the roadmap extends reach.

Each module, walked through the five pillars

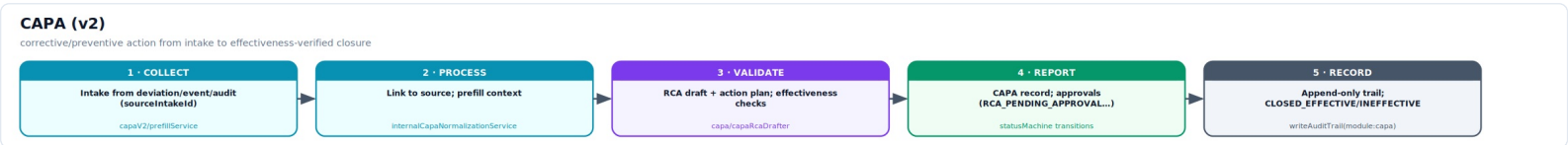
The shared backbone means every module is the same five-pillar motion over the same substrate. Below, each module is shown walking pillar-by-pillar, with the **actual code component** (monospace) that implements each step — so the consistency is demonstrated, not just asserted.

Deviation



An event is **collected** (intake classifier tags type/classification), **processed** (similar-event linking, normalized to the Deviation model), **validated** (5-Why scaffolder drives root cause + impact assessment), **reported** (disposition drafter writes the verdict), and **recorded** — and at close, `createCapaFromDeviation` auto-spawns a CAPA. The deviation's output becomes the next module's input.

CAPA (v2)



A corrective action is **collected** from a deviation/event/audit (carrying `sourceIntakeId`), **processed** (linked + prefilled), **validated** (RCA drafter + effectiveness checks), **reported** (the CAPA record moves through approval states in `statusMachine.js`), and **recorded** (append-only trail; closes EFFECTIVE or INEFFECTIVE). Same five pillars, CAPA content.

Change Control



A change is **collected** (changeNumber, type, requester), **processed** (impact assessment, supplier/site linkage), **validated** (riskLevel + regulatoryImpact + validationRequired against rules), **reported** (multi-step approval with signatures), and **recorded** (trail; may set `triggersRequalification`). Identical motion, change-control content.

Notice the pattern. Each strip is the same five colored stages in the same order. Only the domain fields and the AI feature differ. That sameness *is* the unified architecture — and it's why a new module is mostly new configuration + a domain model, not a new pipeline.

Document Control, Audit, Supplier Quality

Document Control



A document is **collected** (upload/new version), **processed** (classifier + versioning), **validated** (review/approve gates), **reported** (the controlled effective version, e-signed), and **recorded** (version trail) — and `triggerTrainingOnDocumentRevision` auto-assigns retraining to affected users. Cross-module wiring again turns one module's output into another's trigger.

Audit Management



An audit is **collected** (pre-audit questionnaire, evidence, Supplier Intel public data), **processed** (normalized to the audit master with phases/milestones), **validated** (complianceEvaluationService runs responses against the standard's controls + risk), **reported** (cited report + closure certificate, e-signed), and **recorded** (buildSnapshotHash SHA-256 + Part-11 trail). This is the flagship instantiation of all five pillars.

Supplier Quality



Supplier risk is **collected** (public FDA/EMA/WHO data + audit outcomes), **processed** (entity resolution to one supplier identity), **validated** (risk scoring V1/V2 across regulatory/recalls/trend), **reported** (calculateSupplierScorecard), and **recorded** (SupplierRiskSnapshot stored + trail). A continuous, not one-shot, run of the pipeline.

The cross-module fabric. `crossModuleService.js` wires these together so the pillars chain across modules, not just within one: Deviation→CAPA (`createCapaFromDeviation`), Document→Training (`triggerTrainingOnDocumentRevision`), CAPA-failure/Risk→for-cause Audit (`triggerForCauseAudit`), Audit/events→Supplier scorecard (`calculateSupplierScorecard`), and KPI roll-up to Management Review (`aggregateQualityKPIs`). So the whole platform is one large five-pillar engine whose modules feed each other's pillar-1 intake.

The shared backbone & module reference

The walkthroughs above showed each module through the pillars. This page is the underlying reference: the shared backbone every module reuses, and a compact index. Each module is the same backbone — a Model, a status/workflow machine, a service, an AI feature folder, audit-trail writes, and config resolution — differing only in its domain data.

The shared backbone (every module has these)

Layer	Shared mechanism	Evidence
Data model	Mongoose models in <code>src/models/</code>	170 models; each module has its own (DeviationModel, capaV2Models, ChangeControlModel, auditReportModel...).
Workflow / status	Status machine + shared WorkflowDefinition	<code>capaV2/statusMachine.js</code> (transition map + requiredFieldsByStatus); <code>WorkflowDefinitionService</code> (tenant-scoped, by workflowKey).
Service logic	Per-module service + cross-module wiring	<code>eqms/</code> services, <code>crossModuleService.js</code> (deviation→CAPA, doc→training, risk→audit...).
AI feature	Per-module drafter + prompt under <code>ai/features/</code>	audit→observationDrafter; capa→capaRcaDrafter; deviation→fiveWhyScaffolder (+6 more); docControl→documentClassifier.
Audit trail	One writer, all modules	<code>auditTrailService.writeAuditTrail({ module })</code> — module = deviation capa document_control change_control ...
Config	Module toggles + vocabulary + standards	<code>universalModuleConfigService</code> · <code>vocabularyService</code> · <code>standardRegistryService</code> .

The modules, mapped to the same five-pillar flow

Module	Collect (1)	Process (2)	Validate (3)	Report (4) + Record (5)
Audit Mgmt	PAQ, evidence, public data	Normalize to audit model	Compliance run vs controls	Audit report + cert; trail
CAPA (v2)	Intake from deviation/event	Link + prefill	RCA + effectiveness checks	CAPA record; trail
Deviation	Intake classifier	Similar-finder, link	5-Why, disposition	Disposition record; trail
Change Control	Change request intake	Impact linkage	Risk-assess vs rules	Approved change; trail
Doc Control	Document upload	Classify, version	Approval workflow	Controlled version; trail
Supplier Quality	Public + audit data	Entity resolution	Risk scoring	Scorecard/snapshot; trail

This is the heart of the answer to "does every module follow the same architecture?" — yes. Each module is the same five-pillar motion (collect → process → validate → report → record) over the same shared substrate (models + workflow + audit trail + config), differing only in domain content. `crossModuleService` then wires them so one module's output (a deviation) becomes another's input (a CAPA), automatically. That's the unified architecture, confirmed in code.

What verification changed

Confirmed (your mental model holds)

- **Five pillars are real and discrete** — each maps to actual files/directories.
- **Industry-agnostic by design is real** — standards, vocabulary, workflows, and module toggles are all data/config, tenant-resolved.
- **Pluggable AI is real** — multi-LLM gateway, grounded generation, redaction, governance (catalog/cost/permissions/usage), per-module AI features.
- **Every module shares one architecture** — same model+workflow+audit-trail+config backbone; cross-module wiring connects them.
- **Immutable record is real** — SHA-256 snapshot hashing + append-only Part-11/Annex-11 trail + AI decision record.

Refined (where reality differs from the idealized diagram)

- **Two runtimes, not one** — Pillars 1–2 are a Python data-platform; Pillars 3–5 are the Node app. The "one pipeline" spans both.
- **AI attaches per-module and per-wave**, not literally one slot per pillar — the slot metaphor is a clean abstraction over a richer reality.
- **Validation today is audit/compliance-centric** — a genuine rules engine, but instantiated for the audit-questionnaire-vs-standard flow; "validate any supply-chain data" is the to-be generalization.
- **"Immutable" = tamper-evident, not blockchain** — per-record hashing + append-only trail; no chained ledger in code.
- **On-prem LLM = validation-gated, not proven e2e; tool runtime = custom, not MCP.**

The bottom line. Your five-pillar mental model is not just a nice story — it is substantially how the system is actually built. The verification strengthens it: the fundamentals (data platform, rules engine, audit substrate, config layer) genuinely exist, AI is real and pluggable, and every module follows the same architecture. The honest refinements (two runtimes, AI per-module, tamper-evident not blockchain, pharma-proven) make the claims defensible to a technical buyer or investor who will inspect the code — which is exactly the standard that matters.

Provenance of this document. Traced from codex_backend_01 cloned this session (depth-1, default branch). 170 models, the directories and files named throughout are real. Frontend not re-traced here (this focuses on the data/architecture backbone). Re-verify against the live deployed branch before external use, as the repo may differ from production. The two-runtime and per-module findings are from static inspection — confirm runtime behaviour (e.g. that the Python platform feeds the Node app as drawn) with the team.